

Coding Assessment

Author: Soumyajit Nandi**Language:** Java (Java 8+)**Duration:** 90 Minutes**Total Problems:** 5**Focus Area:** Loop Control & Iterative Logic**Total Points:** 100

1. Monotonic Array Validator

Easy

15 pts

An array is **monotonic** if it is either entirely monotone increasing or monotone decreasing.

An array `nums` is monotone increasing if for all $i \leq j$, `nums[i] ≤ nums[j]`. An array `nums` is monotone decreasing if for all $i \leq j$, `nums[i] ≥ nums[j]`.

Given an integer array `nums`, return `true` if the given array is monotonic, or `false` otherwise. Your implementation must execute in a single-pass loop over the elements.

Example 1:**Input:** `nums = [1,2,2,3]`**Output:** `true`**Example 2:****Input:** `nums = [6,5,4,4]`**Output:** `true`**Example 3:****Input:** `nums = [1,3,2]`**Output:** `false`**Constraints:**

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^5 \leq \text{nums}[i] \leq 10^5$

Java

```
class Solution {  
    public boolean isMonotonic(int[] nums) {  
  
    }  
}
```

2. Max Consecutive Ones II

Easy

15 pts

Given a binary array `nums`, return *the maximum number of consecutive 1s in the array if you can flip at most one 0*.

Your logic should dynamically keep track of sliding structural iterations or localized count boundaries without recalculating elements from scratch.

Example 1:

Input: `nums = [1,0,1,1,0]`

Output: 4

Explanation: Flip the first zero to get the maximum number of consecutive 1s. After flipping, the maximum number of consecutive 1s is 4.

Example 2:

Input: `nums = [1,0,1,1,0,1,1,1]`

Output: 6

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- `nums[i]` is either 0 or 1.

Java

```
class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {

    }
}
```

3. Container With Most Water

Medium

20 pts

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `ith` line are `(i, 0)` and `(i, height[i])`.

Find two lines that together with the x-axis form a container, such that the container contains the most water. Return *the maximum amount of water a container can store*.

Notice: You may not slant the container. A valid strategy must bypass an $O(n^2)$ brute-force approach by establishing two convergence looping pointers.

Example 1:

Input: `height = [1,8,6,2,5,4,8,3,7]`

Output: 49

Explanation: The vertical lines are represented by array `[1,8,6,2,5,4,8,3,7]`. The max area of water is 49.

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 10^5$
- $0 \leq \text{height}[i] \leq 10^4$

Java

```
class Solution {  
    public int maxArea(int[] height) {  
  
    }  
}
```

4. Longest Substring with At Most K Distinct Characters

Medium

25 pts

Given a string `s` and an integer `k`, return *the length of the longest substring of `s` that contains at most `k` distinct characters*.

To safely pass all production scaling test cases, design a sliding window loop format to dynamically adjust character distribution maps.

Example 1:

Input: `s = "eceba", k = 2`

Output: 3

Explanation: The substring is "ece" with length 3.

Example 2:

Input: `s = "aa", k = 1`

Output: 2

Explanation: The substring is "aa" with length 2.

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $0 \leq k \leq 50$

Java

```
class Solution {  
    public int lengthOfLongestSubstringKDistinct(String s, int k) {  
  
    }  
}
```

5. Minimum Window Substring Hard

25 pts

Given two strings `s` and `t` of lengths `m` and `n` respectively, return the **minimum window substring** of `s` such that every character in `t` (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

An optimal solution must loop with a linear runtime complexity bound of $O(m + n)$ using coordinated right and left boundary modification rules.

Example 1:

Input: `s = "ADOBECODEBANC", t = "ABC"`

Output: `"BANC"`

Explanation: The minimum window substring `"BANC"` includes 'A', 'B', and 'C' from string `t`.

Example 2:

Input: `s = "a", t = "a"`

Output: `"a"`

Example 3:

Input: `s = "a", t = "aa"`

Output: `""`

Constraints:

- `m == s.length`, `n == t.length`
- $1 \leq m, n \leq 10^5$
- `s` and `t` consist of uppercase and lowercase English letters.

Java

```
class Solution {  
    public String minWindow(String s, String t) {  
  
    }  
}
```